MINI ERP + CRM
Operations Portal

Project Type	Full Stack Web Application
Business Domain	Wholesale / Distribution Operations
Prepared By	Ashlesa Behera
Date	3rd September 2026

1. Project Overview

I built this project as a web application for a wholesale or distribution business. The main goal is to keep customer information, products, stock, and sales operations in one place.

Instead of using many spreadsheets or separate systems, employees can use one portal. Different employees can also get access based on their role.

Main users

- Admin: manages the system and users.
- Sales: works with customers and sales activities.
- Warehouse: works with products and stock.
- Accounts: handles finance/accounting-related operations.

Main objectives

- Secure login and role-based access.
- Customer CRM and follow-up management.
- Product and basic inventory management.
- REST APIs with validation and error handling.
- Responsive React interface.
- PostgreSQL database for storing application data.

The project connects the frontend, backend, database, authentication, and business features into one working portal.

2. Technologies I Used

Technology	Why I used it
React	To build the interactive frontend.
TypeScript	To make the frontend and backend more type-safe.
Vite	To run and build the React frontend quickly.
Node.js	To run the backend.
Express.js	To create REST API routes and backend logic.
PostgreSQL / Neon	To store application data.
Prisma	To work with PostgreSQL using an ORM.
JWT	To authenticate users and protect API routes.
Zod	To validate incoming data.
Axios	To connect the React frontend with the backend API.
Git / GitHub	To manage and store source code.

How the parts connect

React + TypeScript + Vite → REST API → Node.js + Express.js → Prisma → PostgreSQL / Neon

Simple idea: the frontend sends requests, the backend processes them, Prisma communicates with the database, and the database stores the data.

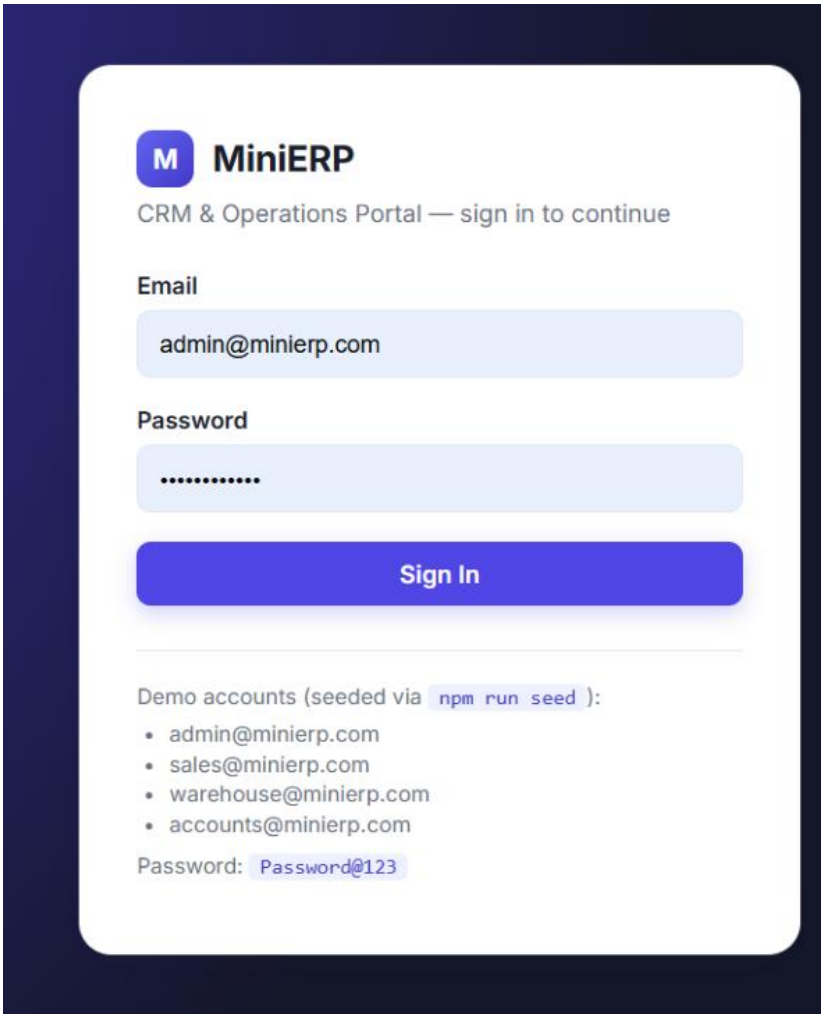
3. Login and Role-Based Access

The application uses JWT-based authentication. The user enters an email and password. The backend checks the credentials and returns a JWT token when the login is successful.

After login, the frontend uses the token for protected API requests. The backend checks the token before allowing protected operations.

Login flow: Enter login details → Backend validates → JWT token returned → Protected pages and APIs

The project defines four roles: Admin, Sales, Warehouse, and Accounts. Role checks can be used to control sensitive operations.

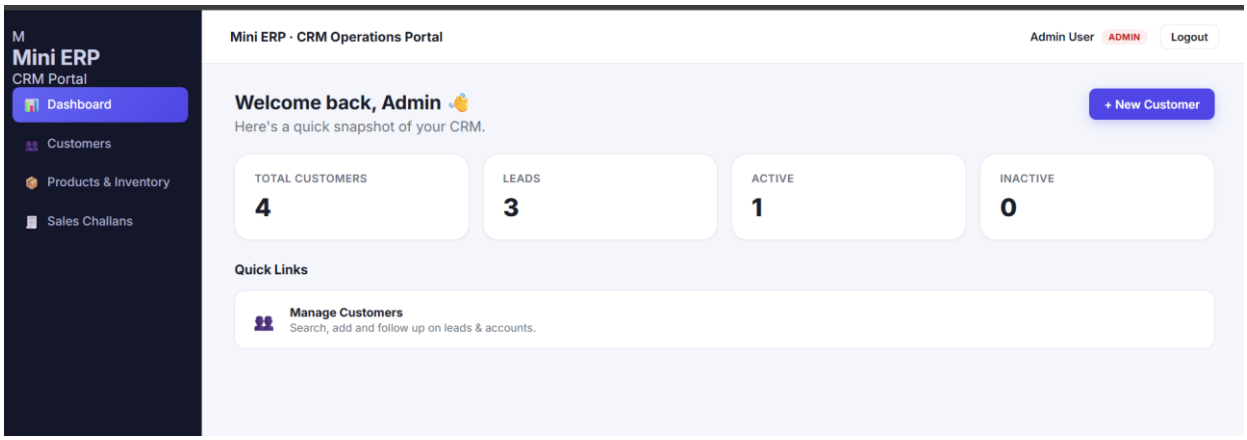


Screenshot 1 — Login page

4. Dashboard

The dashboard gives a quick view of the CRM. It shows the number of customers, leads, active customers, and inactive customers.

There are also quick links to important operations such as customer management.



Screenshot 2 — Admin dashboard

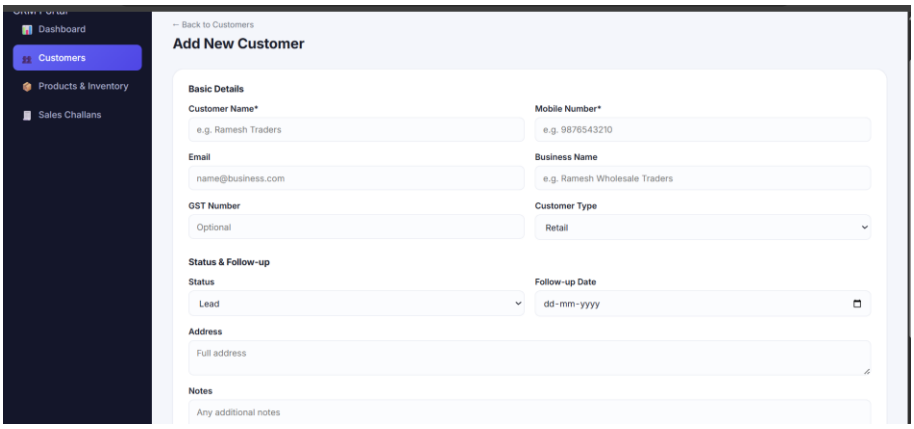
5. Customer CRM Module

The Customer module is used to store and manage customer information. It helps the sales team search customers, view details, track status, and plan follow-ups.

Important customer fields include customer name, mobile number, email, business name, GST number, customer type, address, status, follow-up date, and notes.

Customer features

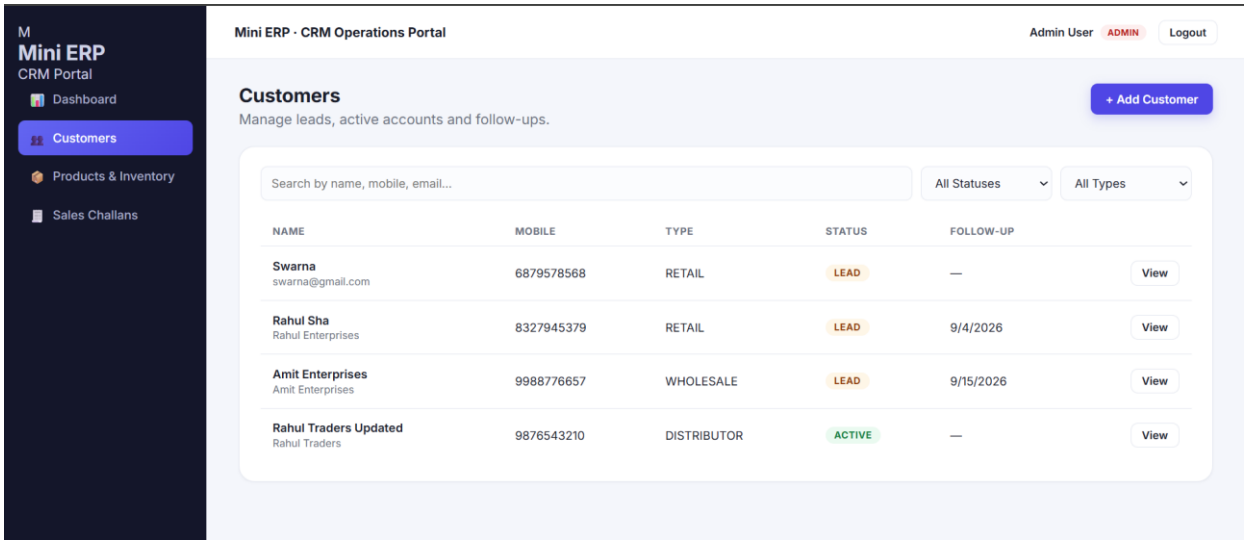
- Add a customer.
- Edit customer information.
- Search customers.
- View customer details.
- Add follow-up notes.
- Track customer status and customer type.



Screenshot 3 — Add New Customer page

6. Customer List and Search

The customer list shows important information in one table. The user can search by name, mobile number, or email and can filter by status and customer type.



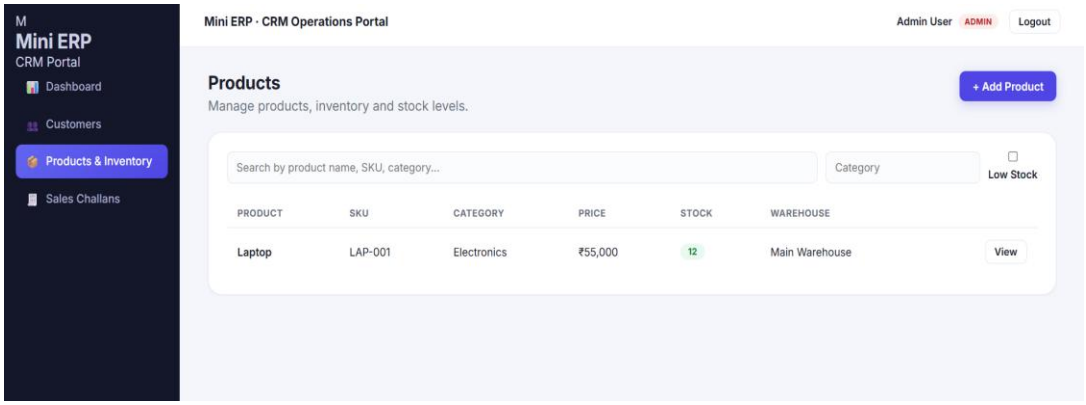
Screenshot 4 — Customer list / CRM page

This screen demonstrates the CRM idea clearly: customer data is organized, searchable, and easy for the sales team to follow.

7. Product and Inventory Module

This module stores product information and basic stock details. It is mainly useful for warehouse and operations work.

Important fields include product name, SKU, category, unit price, current stock, minimum stock, and warehouse location.



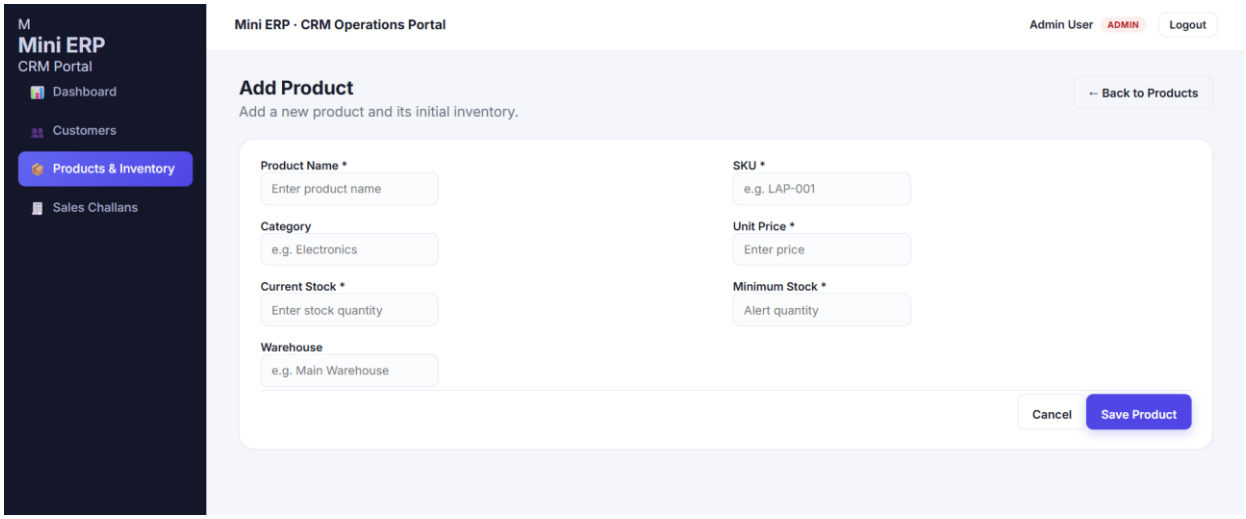
Screenshot 5 — Product and Inventory list

The screen supports searching products and checking stock levels. The documentation also describes low-stock monitoring as part of the module.

8. Adding a Product

The Add Product screen is used to create a product and enter its initial inventory information.

The user can enter product name, SKU, category, unit price, current stock, minimum stock, and warehouse.



Screenshot 6 — Add Product page

9. Sales Challan Workflow

The original documentation had a placeholder for the Sales Challan screenshot. Since no Sales Challan screenshot was provided, I replaced that placeholder with a simple workflow.

Select customer → Add products and quantity → Save challan → Confirm sale → Reduce stock

Important business rules

- A draft challan should not reduce stock.
- Confirming a challan should reduce stock.
- Stock should never become negative.
- If stock is insufficient, the API should return a clear error.
- Challan items should keep product snapshot information.

Status note: the source documentation says the exact completion of the Sales Challan module should be verified against the current repository. Therefore, this guide does not claim the full challan workflow is complete.

10. Backend, API and Database

The frontend communicates with the backend using REST APIs. The backend handles authentication, validation, business logic, authorization, and database operations.

Prisma acts as the database access layer between the Express backend and PostgreSQL.

Examples of API routes

Method	Endpoint	Purpose
POST	/api/auth/login	Login and return JWT
GET	/api/auth/me	Get current user
GET	/api/customers	List/search customers
POST	/api/customers	Create customer
GET	/api/customers/:id	Get customer details
PUT	/api/customers/:id	Update customer
POST	/api/customers/:id/followups	Add follow-up note
GET	/api/products	List products, if implemented
POST	/api/products	Create product, if implemented

Validation and error handling are also part of the design. Invalid input, unauthorized access, missing resources, insufficient stock, and unexpected server/database errors should return controlled responses.

11. Setup, Configuration and Security

For local development, the project needs Node.js, npm, Git, PostgreSQL or Neon PostgreSQL, and a code editor such as Visual Studio Code.

Backend setup: install dependencies, configure the backend .env file, configure the database and JWT secret, run Prisma generation/migrations, and start the development server.

Frontend setup: install dependencies, configure VITE_API_URL, and start the Vite development server.

Local URLs in the source documentation

- Frontend: http://localhost:5173
- Backend: http://localhost:5000
- Backend API: http://localhost:5000/api

Security points

- Passwords should not be stored in plain text.
- JWT secrets and database connection strings should stay in environment variables.
- Protected endpoints should require authentication.
- Sensitive operations should use role checks.
- Input should be validated before database operations.
- Production CORS should allow only trusted frontend origins.

12. Current Status and Future Improvements

The source documentation says authentication is implemented, the core Customer CRM is implemented, and basic Product/Inventory work is implemented. It also says the full Sales Challan workflow should be verified before submission.

Features listed as optional or future improvements include invoices, purchase orders, PDF export, product image upload, Docker, CI/CD, automated tests, audit logging, and dashboard charts.

A short interview explanation

“I developed a Mini ERP and CRM Operations Portal for a wholesale and distribution business. The main purpose was to manage customers, products, inventory, and sales operations in one application. I used React with TypeScript and Vite for the frontend, Node.js with Express and TypeScript for the backend, PostgreSQL with Prisma for the database, JWT for authentication, Zod for validation, and Axios for API communication. I implemented role-based access for Admin, Sales, Warehouse, and Accounts users. The Customer CRM supports customer management, search, status, and follow-ups, while the Product module manages product and basic stock information. I also designed REST APIs with validation and controlled error handling.”